

AUTOSAR Document Intelligence Assistant

RAG-Powered ML Software Engineering Assignment Report

Course: AIMLCZG546 - Software Engineering for Machine Learning

Assignment I | Group 151 | BITS Pilani WILP

Submission artifacts: 151.docx, 151.pdf, 151.ipynb, source code repository

Submission readiness statement

This report consolidates the required project details, GR4ML requirement views, system architecture, selected architectural patterns, implementation evidence, screenshots, code references, testing evidence, and contribution details for Group 151.

#	BITS ID	Name	Qualitative Contribution	%
1	2025aa05473	Abhinav Mandloi	Project integration, FastAPI gateway, report packaging, final verification	25
2	2025aa05564	Prajwal Rastogi	Ingestion pipeline, PDF parsing, chunking, data preparation view	20
3	2025aa05686	Prithish Joshi	Retrieval pipeline, vector search, prompt/citation design	20
4	2025aa05553	Satwinder Singh	Quality requirements, health monitoring, heartbeat, reliability checks	18
5	2025aa05533	Shray Vijay	UI, feedback loop, tests, screenshots, documentation support	17

1. Executive Summary

The AUTOSAR Document Intelligence Assistant is a local Retrieval-Augmented Generation (RAG) system for querying AUTOSAR Basic Software specification PDFs. It ingests PDF documents, extracts and chunks technical content, embeds chunks with a local neural embedding model, indexes them in ChromaDB, and answers natural language questions with source citations through a FastAPI application and chat UI.

- Domain: Automotive embedded software and AUTOSAR Basic Software specifications.
- ML task: semantic retrieval plus grounded answer generation using local Ollama models.
- Current indexed corpus: 5 AUTOSAR PDFs, 2,717 pages, 4,768 chunks.
- Verification: healthy service checks, passing focused tests, API search smoke test, and live UI answer screenshot.

2. Assignment Requirement Coverage

Assignment item	Evidence in submission
Problem statement and ML domain	Defined in README, GR4ML_Requirements.md, and this report.
Measurable goals and requirement specifications	GR4ML goals, functional requirements, and non-functional requirements documented.
GR4ML Business View	docs/Business_View.md plus summarized in this report.
GR4ML Analytics Design View	docs/Analytics_Design_View.md plus summarized in this report.
GR4ML Data Preparation View	docs/Data_Preparation_View.md plus ingestion implementation.
Top three quality requirements	Accuracy, explainability, and availability documented with rationale.
System architecture diagram	HLD.png embedded below.
Two architectural patterns	RAG pattern and service-oriented/API-gateway decomposition with heartbeat/circuit breaker.
Implementation using technologies/tools	FastAPI, ChromaDB, Ollama, PyMuPDF, static chat UI, pytest.
Screenshots of application	UI, API docs, and generated answer screenshots included.
Code and contribution details	Code references and group contribution table included.

3. Problem Statement and ML Justification

AUTOSAR specifications are large, cross-referenced, and written in precise technical language. Engineers often need to locate exact configuration parameters, APIs, requirement IDs, and module relationships across multiple PDFs. Manual lookup can take 15-30 minutes per question and can miss relevant sections.

The selected ML solution is appropriate because semantic search is a relevance prediction task. Learned embeddings can match user questions to relevant specification chunks even when the wording differs, and the LLM can synthesize a readable answer from retrieved context while citations preserve traceability.

Goal ID	Metric	Target / evidence
G1	Retrieval Precision@5	Target $\geq 85\%$; designed through semantic chunking, embeddings, and

		reranking.
G2	Retrieval latency	/query/search observed at 29-428 ms in smoke tests; full /query reports local LLM latency.
G3	Hallucination control	Prompt guardrails plus citations; unsupported claims target < 5%.
G4	Citation accuracy	Responses return document, page, section, and source chunks.
G5	Ingestion throughput	Large System Template PDF ingested in 278.75 seconds for 2,261 pages.
G6	Availability	Health and heartbeat endpoints check Ollama, models, and vector store.

4. GR4ML Views

4.1 Business View

Primary actors are embedded systems engineers, AUTOSAR integrators, technical leads, and QA/testing teams. The business goal is to query AUTOSAR specifications quickly and reliably so engineering decisions can be traced back to official source content.

1. Decision goal: decide the correct technical implementation from AUTOSAR specification evidence.
2. Question goal: identify the relevant specification content for a user query.
3. Insight: the RAG system generates citation-backed answers and source snippets.

4.2 Analytics Design View

The analytics goal is semantic retrieval and answer generation for AUTOSAR questions. The design uses nomic-embed-text for embeddings, cosine similarity/HNSW search through ChromaDB, optional LLM reranking, prompt construction, and local llama3.2 generation.

4.3 Data Preparation View

The data pipeline parses PDFs, detects section-like headings and requirement IDs, chunks content by semantic boundaries, generates dense embeddings, and stores vector plus metadata records for citation-backed retrieval.

Operator	Implementation file	Purpose
PDF parsing	app/services/ingestion/parser.py	Extract text, page metadata, headings, and requirement IDs.
Semantic chunking	app/services/ingestion/chunker.py	Split by sections and token limits with overlap.
Embedding	app/services/ingestion/embedder.py	Generate local Ollama embeddings.
Vector indexing	app/storage/vector_store.py	Persist searchable vectors in ChromaDB.
Metadata store	app/storage/metadata_store.py	Track ingested documents and chunking parameters.

5. Top Three Quality Requirements

Quality requirement	Why it matters	Architectural response
---------------------	----------------	------------------------

Accuracy	Wrong AUTOSAR guidance can lead to incorrect embedded software decisions.	Section-aware chunking, embeddings, reranking, and source metadata.
Explainability	Engineers must verify answers against official specifications.	Citations include document, page, section, and source chunks.
Availability	The tool should not fail silently when local models or vector storage are unavailable.	Heartbeat checks, health endpoints, retry logic, and circuit breaker behavior.

6. System Architecture

The architecture separates one-time ingestion from runtime retrieval and generation. It also identifies ML and non-ML components, API surfaces, storage, feedback, and cross-cutting concerns.

AUTOSAR Document Intelligence Assistant

RAG-Powered Architecture (High-Level Design)

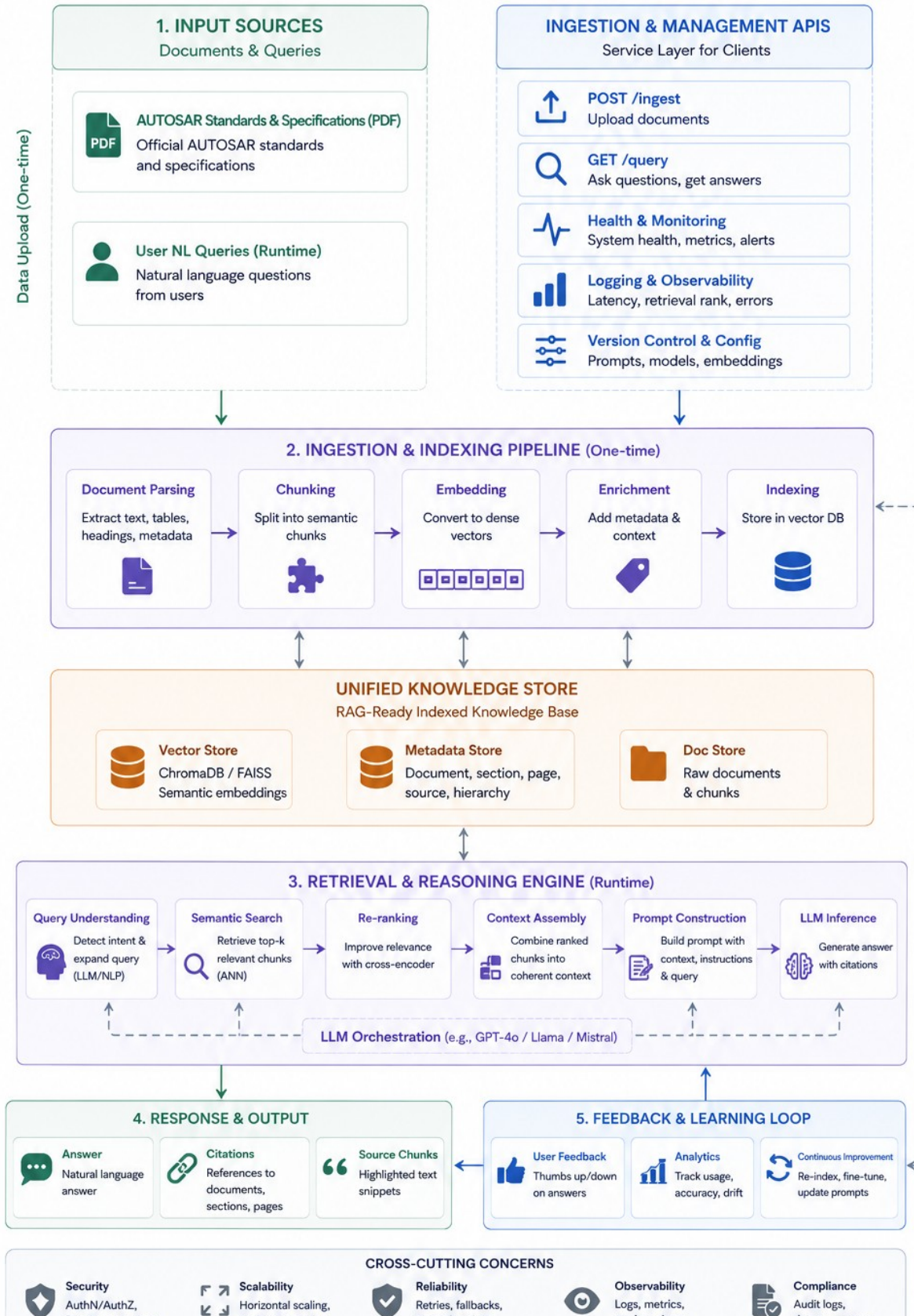


Figure 1. High-level architecture diagram showing ingestion, vector store, retrieval, response, feedback, and cross-cutting concerns.

7. Architectural Patterns Implemented

7.1 Pattern 1: Retrieval-Augmented Generation

RAG decouples domain knowledge from the LLM. AUTOSAR content is indexed in a vector store, relevant chunks are retrieved for a query, and the LLM is constrained to answer from the retrieved context with citations.

```
# app/services/retrieval/router.py
search_results = await semantic_search(query=request.question, top_k=retrieval_top_k)
ranked_results = await rerank(query=request.question, search_results=search_results,
top_n=request.top_k)
context = assemble_context(ranked_results)
answer = await generate_completion(prompt=query_prompt, system_prompt=system_prompt)
citations = extract_citations(answer, ranked_results)
```

7.2 Pattern 2: Service-Oriented API Gateway with Heartbeat Reliability Tactic

The FastAPI application exposes bounded service routers for ingestion, retrieval, feedback, and monitoring. Cross-cutting middleware adds correlation IDs, and the heartbeat checks dependencies such as Ollama, the embedding model, the LLM model, and ChromaDB.

```
# app/main.py
app.include_router(health_router)
app.include_router(ingestion_router)
app.include_router(retrieval_router)
app.include_router(feedback_router)

# app/monitoring/health.py
await asyncio.gather(
    check_ollama_api(), check_embedding_model(), check_llm_model(), check_vector_store()
)
```

8. Implementation Summary

Layer	Technology	Implemented capability
Frontend	Static HTML/CSS/JS	Chat UI, health status, feedback controls, source display.
API	FastAPI + uvicorn	REST endpoints, middleware, docs, global error handling.
Ingestion	PyMuPDF + custom chunker	PDF parsing, chunk creation, metadata enrichment.
ML embeddings	Ollama nomic-embed-text	Dense vector representations for document chunks and queries.
Vector DB	ChromaDB	Persistent similarity search over AUTOSAR chunks.
LLM	Ollama llama3.2	Local grounded answer generation and reranking.
Monitoring	Health/metrics routers	Heartbeat, dependency checks, latency and usage metrics.
Testing	pytest	Parser, chunker, citations, health, error handling, UI regression, RAG checks.

9. Application Screenshots

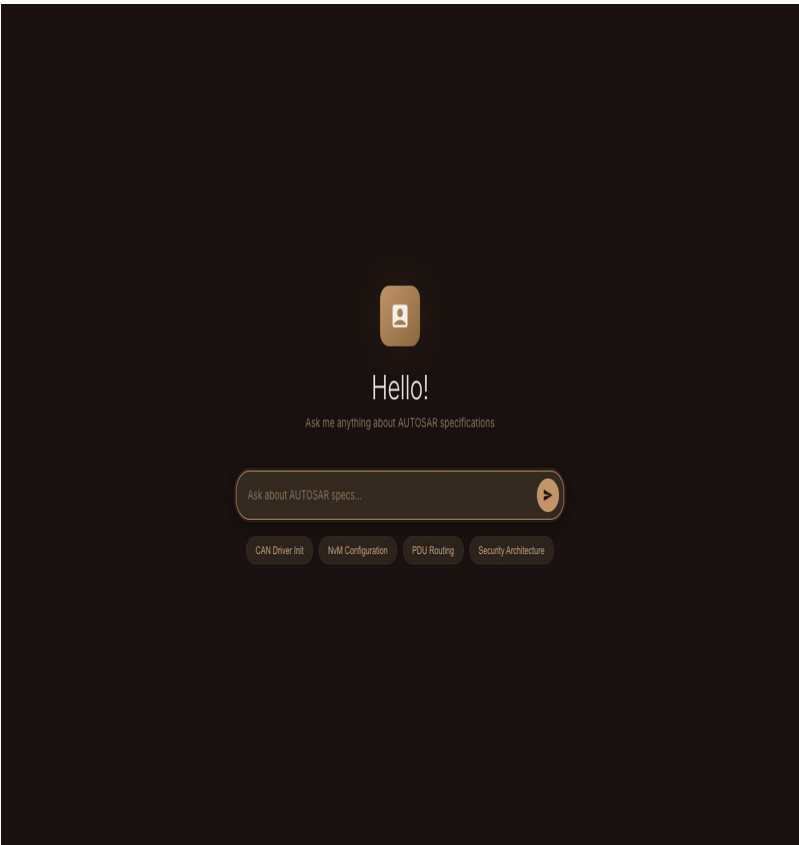
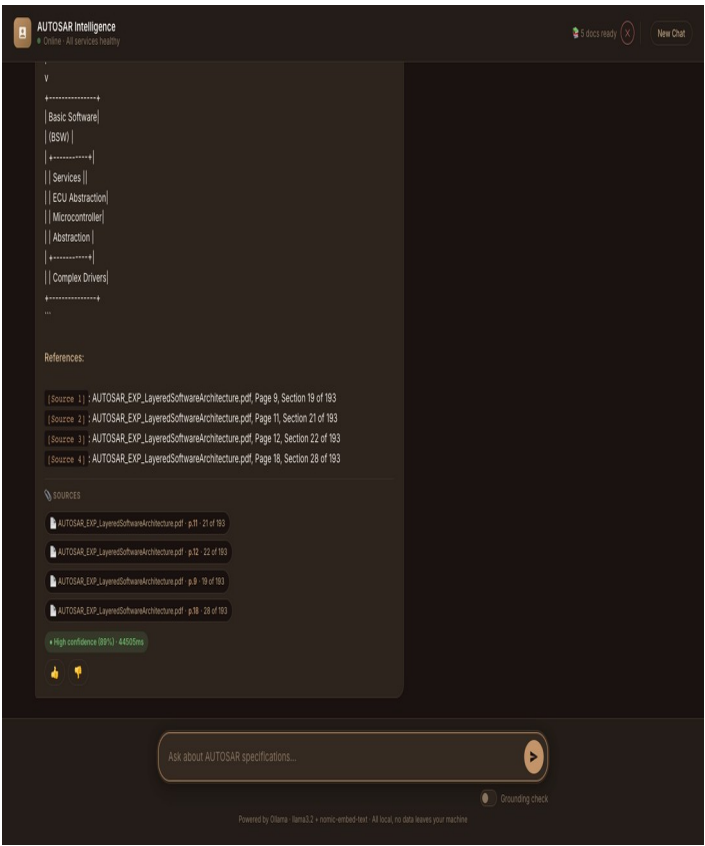


Figure 2. Chat UI home screen with healthy local service status.



AUTOSAR Document Intelligence Assistant

1.0.0 OAS 3.1

openapi.json

RAG-powered system for intelligent querying of AUTOSAR BSW specifications. Upload AUTOSAR PDFs, ask natural language questions, and get precise answers with citations. Built with Ollama (local LLM), ChromaDB (vector store), and FastAPI.

Health & Monitoring

GET	/health/status	Health Status	▼
GET	/health	Health Check	▼
GET	/health/metrics	Get Metrics	▼

Ingestion Service

POST	/ingest/upload	Ingest Upload	▼
POST	/ingest/local	Ingest Local	▼
GET	/ingest/documents	List Documents	▼
DELETE	/ingest/documents/{document_name}	Delete Document	▼

Retrieval Service

POST	/query	Query Documents	▼
POST	/query/search	Search Only	▼

Feedback & Learning Loop

POST	/feedback	Submit Feedback	▼
GET	/feedback/analytics	Get Analytics	▼

UI

GET	/	Serve UI	▼
-----	---	----------	---

Root

GET	/api	Api Overview	▼
-----	------	--------------	---

Schemas

Body_ingest_upload_ingest_upload_post	Expand all	object
Citation	Expand all	object
ClaimEvaluation	Expand all	object
DocumentInfo	Expand all	object
FeedbackRequest	Expand all	object
FeedbackResponse	Expand all	object
HTTPValidationError	Expand all	object
HallucinationEvaluation	Expand all	object
IngestionResponse	Expand all	object
LocalIngestionRequest	Expand all	object
QueryRequest	Expand all	object
QueryResponse	Expand all	object
SearchOnlyResponse	Expand all	object
ValidationError	Expand all	object

Figure 3. FastAPI OpenAPI documentation showing health, ingestion, retrieval, feedback, UI, and root endpoints.

10. Verification Evidence

Dependency	Status	Last healthy
ollama_api	healthy	2026-07-02T16:25:27.725759+00:00
embedding_model	healthy	2026-07-02T16:25:27.758559+00:00
llm_model	healthy	2026-07-02T16:25:28.018646+00:00
vector_store	healthy	2026-07-02T16:25:27.722577+00:00

Ingestion registry evidence:

Document	Pages	Chunks	Ingestion time
AUTOSAR_CP_SWS_CANDriver.pdf	131	531	15.07 s
AUTOSAR_CP_TPS_SystemTemplate.pdf	2261	3518	278.75 s
AUTOSAR_EXP_LayeredSoftwareArchitecture.pdf	182	209	9.76 s
AUTOSAR_EXP_SecurityOverview.pdf	26	86	3.72 s
AUTOSAR_SWS_EthernetDriver.pdf	117	424	17.18 s

Retrieval smoke test evidence:

Question	Results	Latency	Top source
What is AUTOSAR layered software architecture?	3	178.5 ms	AUTOSAR_EXP_LayeredSoftwareArchitecture.pdf

Latency note

The system intentionally uses local Ollama models for privacy and zero external API cost. Search-only retrieval is fast on the prepared vector store. Complete natural-language answer generation is local-LLM dependent and exposes latency_ms in the API/UI; observed full-answer runs ranged from about 21.7s to 44.5s on this machine.

11. Testing Summary

- Focused verification: 12/12 UI regression and non-expensive API tests passed.
- Broader pytest run: 51 tests passed before stopping the expensive full-ingestion integration path that reprocesses all PDFs.
- Notebook code cells were executed successfully against the local server: health, metadata, and query/search checks passed.
- Live UI screenshot captured a generated answer with citations from AUTOSAR_EXP_LayeredSoftwareArchitecture.pdf.

12. Final Submission Checklist

Item	Status
151.docx final report	Created
151.pdf final report	Created from DOCX via LibreOffice

151.ipynb notebook	Created and code cells validated
Group member details and contribution percentages	Included and totals 100%
Screenshots of application	Included
Architecture diagram	Included
Code references and snippets	Included
README group/member consistency	Fixed
README HLD image link	Fixed
Latency claims	Updated to measured retrieval/full-generation behavior

Deadline note

The assignment brief lists the due date as 30 June 2026, 23:00. This report cannot change the portal timestamp; it makes the local project and submission artifacts complete and consistent for upload or resubmission.